

- **University Name: Charutar Vidya Mandal University (CVMU)**
- **Collage Name: A.D. Patel Institute of Technology (ADIT)**
- **Course Title & code: Programing with JAVA (202044502)**
- **Assignment Number: Assignment 2**
- **Student Name: Patel Yashkumar Shaileshkumar**
- **Enrollment Number: 12402080601158**
- **Semester & academic Year: sem-4 (2025-26)**
- **Github Link: <https://github.com/yash-1158/Java-Programming-Assignment>**

Assignment 2

➤ Program 1

Problem Statement:

Find prime numbers using multithreading (Thread, Runnable, Executor Framework)

Objective:

To develop a Java program that finds prime numbers using multithreading by implementing different approaches such as Thread class, Runnable interface, and Executor Framework to demonstrate concurrent execution.

Source Code:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Program1 {
    static class PrimeThread extends Thread {
        int start, end;
        PrimeThread(int s, int e) {
            start = s;
            end = e;
        }
        boolean isPrime(int n) {
            if (n <= 1) return false;
            for (int i = 2; i <= n / 2; i++)
                if (n % i == 0) return false;
            return true;
        }
        public void run() {
            StringBuilder primes = new StringBuilder();
            for (int i = start; i <= end; i++)
                if (isPrime(i)) primes.append(i).append(" ");
            synchronized (System.out) {
                System.out.println("Thread Class: Prime numbers from " + start + " to " + end + " :");
                System.out.println(primes + "\n");
            }
        }
    }

    static class PrimeRunnable implements Runnable {
        int start, end;
        PrimeRunnable(int s, int e) {
            start = s;
            end = e;
        }
        boolean isPrime(int n) {
            if (n <= 1) return false;
            for (int i = 2; i <= n / 2; i++)
                if (n % i == 0) return false;
            return true;
        }
        public void run() {
            StringBuilder primes = new StringBuilder();
            for (int i = start; i <= end; i++)
                if (isPrime(i)) primes.append(i).append(" ");
            synchronized (System.out) {
                System.out.println("Runnable Interface: Prime numbers from " + start + " to " + end + " :");
                System.out.println(primes + "\n");
            }
        }
    }

    static class PrimeTask implements Runnable {
        int start, end;
        PrimeTask(int s, int e) {
            start = s;
            end = e;
        }
        boolean isPrime(int n) {
            if (n <= 1) return false;
            for (int i = 2; i <= n / 2; i++)
                if (n % i == 0) return false;
            return true;
        }
    }
}
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

```

    if (n <= 1) return false;
    for (int i = 2; i <= n / 2; i++)
        if (n % i == 0) return false;
    return true;
}

public void run() {
    StringBuilder primes = new StringBuilder();
    for (int i = start; i <= end; i++)
        if (isPrime(i)) primes.append(i).append(" ");
    synchronized (System.out) {
        System.out.println("Executor Framework: Prime numbers from " + start + " to " + end + " :");
        System.out.println(primes + "\n");
    }
}

}

public static void main(String[] args) {
    System.out.println("=== Prime Numbers Using Multithreading ===\n");
    PrimeThread t1 = new PrimeThread(1, 50);
    PrimeThread t2 = new PrimeThread(51, 100);
    t1.start();
    t2.start();
    try {
        t1.join();
        t2.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    Thread r1 = new Thread(new PrimeRunnable(1, 50));
    Thread r2 = new Thread(new PrimeRunnable(51, 100));
    r1.start();
    r2.start();
    try {
        r1.join();
        r2.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    ExecutorService executor = Executors.newFixedThreadPool(2);
    executor.execute(new PrimeTask(1, 50));
    executor.execute(new PrimeTask(51, 100));
    executor.shutdown();
    try {
        executor.awaitTermination(2, TimeUnit.MINUTES);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println("=== All Tasks Completed ===");
}

```

OutPut:

```

=== Prime Numbers Using Multithreading ===

Thread Class: Prime numbers from 1 to 50 :
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Thread Class: Prime numbers from 51 to 100 :
53 59 61 67 71 73 79 83 89 97

Runnable Interface: Prime numbers from 1 to 50 :
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Runnable Interface: Prime numbers from 51 to 100 :
53 59 61 67 71 73 79 83 89 97

Executor Framework: Prime numbers from 1 to 50 :
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Executor Framework: Prime numbers from 51 to 100 :
53 59 61 67 71 73 79 83 89 97

=== All Tasks Completed ===

```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

➤ Program 2

Problem Statement:

Producer–Consumer problem using synchronization and inter-thread communication

Objective:

To develop a Java program that demonstrates the Producer–Consumer problem using synchronization and inter-thread communication to ensure proper coordination between producer and consumer threads.

Source Code:

```
import java.util.LinkedList;
import java.util.Queue;

class SharedBuffer {
    private final Queue<Integer> buffer = new LinkedList<>();
    private final int capacity;
    SharedBuffer(int capacity) {
        this.capacity = capacity;
    }
    public synchronized void produce(int item) throws InterruptedException {
        while (buffer.size() == capacity) wait();
        buffer.add(item);
        System.out.println(Thread.currentThread().getName() + " produced: " + item);
        notifyAll();
    }
    public synchronized int consume() throws InterruptedException {
        while (buffer.isEmpty()) wait();
        int item = buffer.poll();
        System.out.println(Thread.currentThread().getName() + " consumed: " + item);
        notifyAll();
        return item;
    }
}

class Producer implements Runnable {
    private final SharedBuffer buffer;
    Producer(SharedBuffer buffer) {
        this.buffer = buffer;
    }
    public void run() {
        for (int i = 1; i <= 10; i++) {
            try {
                buffer.produce(i);
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
    }  
}  
  
class Consumer implements Runnable {  
    private final SharedBuffer buffer;  
  
    Consumer(SharedBuffer buffer) {  
        this.buffer = buffer;  
    }  
  
    public void run() {  
        for (int i = 1; i <= 10; i++) {  
            try {  
                buffer.consume();  
                Thread.sleep(150);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}  
  
public class Program2{  
    public static void main(String[] args) {  
        SharedBuffer buffer = new SharedBuffer(5);  
  
        Thread producer1 = new Thread(new Producer(buffer), "Producer-1");  
        Thread producer2 = new Thread(new Producer(buffer), "Producer-2");  
  
        Thread consumer1 = new Thread(new Consumer(buffer), "Consumer-1");  
        Thread consumer2 = new Thread(new Consumer(buffer), "Consumer-2");  
  
        producer1.start();  
        producer2.start();  
        consumer1.start();  
        consumer2.start();  
    }  
}
```

length: 2318 lines: 75 ln: 40

Output:

```
C:\Users\adit\Desktop\Java Program>
Producer-1 produced: 1
Consumer-2 consumed: 1
Producer-2 produced: 1
Consumer-1 consumed: 1
Producer-1 produced: 2
Producer-2 produced: 2
Consumer-1 consumed: 2
Consumer-2 consumed: 2
Producer-1 produced: 3
Producer-2 produced: 3
Producer-1 produced: 4
Producer-2 produced: 4
Consumer-1 consumed: 3
Consumer-2 consumed: 3
Producer-1 produced: 5
Producer-2 produced: 5
Consumer-2 consumed: 4
Consumer-1 consumed: 4
Producer-2 produced: 6
Producer-1 produced: 6
Producer-2 produced: 7
Consumer-1 consumed: 5
Consumer-2 consumed: 5
Producer-1 produced: 7
Producer-2 produced: 8
Consumer-1 consumed: 6
Producer-1 produced: 8
Consumer-2 consumed: 6
Producer-2 produced: 9
Consumer-1 consumed: 7
Producer-1 produced: 9
Consumer-2 consumed: 7
Producer-2 produced: 10
Consumer-1 consumed: 8
Producer-1 produced: 10
Consumer-2 consumed: 8
Consumer-2 consumed: 9
Consumer-1 consumed: 9
Consumer-2 consumed: 10
Consumer-1 consumed: 10
```

➤ Program 3

Problem Statement:

Implement CRUD operations using Collection API (ArrayList, HashMap, TreeMap)

Objective:

- To understand and implement CRUD operations in object-oriented programming using Java Collections.
- To learn the difference between ArrayList, HashMap, and TreeMap in terms of data storage, access, and ordering

Source Code:

```
import java.util.*;
public class Program3 {
    public static void main(String[] args) {
        System.out.println("=== CRUD Operations using ArrayList ===");
        ArrayList<String> list = new ArrayList<>();
        list.add("John");
        list.add("Alice");
        list.add("Bob");
        System.out.print("After Adding: {");
        for (int i = 0; i < list.size(); i++) {
            System.out.print((i + 1) + "=" + list.get(i));
            if (i != list.size() - 1) System.out.print(", ");
        }
        System.out.println("}");
        System.out.println("Read Elements:");
        for (int i = 0; i < list.size(); i++) {
            System.out.println((i + 1) + " : " + list.get(i));
        }
        list.set(1, "Alicia");
        System.out.print("After Update: {");
        for (int i = 0; i < list.size(); i++) {
            System.out.print((i + 1) + "=" + list.get(i));
            if (i != list.size() - 1) System.out.print(", ");
        }
        System.out.println("}");
        list.remove(0);
        System.out.print("After Delete: {");
        for (int i = 0; i < list.size(); i++) {
            System.out.print((i + 1) + "=" + list.get(i));
            if (i != list.size() - 1) System.out.print(", ");
        }
        System.out.println("}");
    }
}
```

```
System.out.println("\n=== CRUD Operations using HashMap ===");
HashMap<Integer, String> hmap = new HashMap<>();

hmap.put(1, "John");
hmap.put(2, "Alice");
hmap.put(3, "Bob");

System.out.println("After Adding: " + hmap);

System.out.println("Read Elements:");
for (Map.Entry<Integer, String> entry : hmap.entrySet()) {
    System.out.println(entry.getKey() + " : " + entry.getValue());
}

hmap.remove(1);
System.out.println("After Delete: " + hmap);

System.out.println("\n=== CRUD Operations using TreeMap ===");
TreeMap<Integer, String> tmap = new TreeMap<>();

tmap.put(1, "Red");
tmap.put(3, "Green");
tmap.put(2, "Blue");
System.out.println("After Adding: " + tmap);

System.out.println("Read Elements:");
for (Map.Entry<Integer, String> entry : tmap.entrySet()) {
    System.out.println(entry.getKey() + " : " + entry.getValue());
}

tmap.put(3, "Yellow");
System.out.println("After Update: " + tmap);

tmap.remove(1);
System.out.println("After Delete: " + tmap);
}
```

OutPut:

```
=== CRUD Operations using ArrayList ===
After Adding: {1=John, 2=Alice, 3=Bob}
Read Elements:
1 : John
2 : Alice
3 : Bob
After Update: {1=John, 2=Alicia, 3=Bob}
After Delete: {1=Alicia, 2=Bob}

=== CRUD Operations using HashMap ===
After Adding: {1=John, 2=Alice, 3=Bob}
Read Elements:
1 : John
2 : Alice
3 : Bob
After Delete: {2=Alice, 3=Bob}

=== CRUD Operations using TreeMap ===
After Adding: {1=Red, 2=Blue, 3=Green}
Read Elements:
1 : Red
2 : Blue
3 : Green
After Update: {1=Red, 2=Blue, 3=Yellow}
After Delete: {2=Blue, 3=Yellow}
```


➤ Program 4

Problem Statement:

Sort Book objects using Comparable and Comparator interfaces

Objective:

- To design a Java program for sorting Book objects.
- To implement natural sorting using Comparable interface.
- To apply custom sorting using Comparator interface.
- To demonstrate sorting based on different attributes of Book objects.

Source Code:

```
import java.util.*;
class Book implements Comparable<Book> {
    String title;
    String author;
    double price;
    Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }
    @Override
    public int compareTo(Book other) {
        return Double.compare(this.price, other.price);
    }
}
class TitleComparator implements Comparator<Book> {
    @Override
    public int compare(Book b1, Book b2) {
        return b1.title.compareTo(b2.title);
    }
}
public class Program4 {
    public static void main(String[] args) {
        ArrayList<Book> books = new ArrayList<>();

        books.add(new Book("Java Basics", "Alice", 450.0));
        books.add(new Book("Python Guide", "Bob", 400.0));
        books.add(new Book("Data Structures", "Charlie", 500.0));
        books.add(new Book("Algorithms", "David", 350.0));

        System.out.println("=== Original List ===");
        printBooks(books);
    }
}
```

```

Collections.sort(books);
System.out.println("\n=== Sorted by Price (Comparable) ===");
printBooks(books);

books.sort(new TitleComparator());
System.out.println("\n=== Sorted by Title (Comparator) ===");
printBooks(books);
}

static void printBooks(ArrayList<Book> books) {
    System.out.printf("%-20s %-15s %-8s\n", "Title", "Author", "Price (₹)");
    System.out.println("-----");
    for (Book b : books) {
        System.out.printf("%-20s %-15s ₹%-7.2f\n", b.title, b.author, b.price);
    }
}
}

```

Output:

```

=== Original List ===
Title                Author                Price (?)
-----
Java Basics         Alice                ?450.00
Python Guide         Bob                  ?400.00
Data Structures      Charlie              ?500.00
Algorithms           David                ?350.00

=== Sorted by Price (Comparable) ===
Title                Author                Price (?)
-----
Algorithms           David                ?350.00
Python Guide         Bob                  ?400.00
Java Basics         Alice                ?450.00
Data Structures      Charlie              ?500.00

=== Sorted by Title (Comparator) ===
Title                Author                Price (?)
-----
Algorithms           David                ?350.00
Data Structures      Charlie              ?500.00
Java Basics         Alice                ?450.00
Python Guide         Bob                  ?400.00

```

➤ Program 5

Problem Statement:

Count word occurrences from a file using File Handling APIs

Objective:

- To design a Java program to read data from a file using File Handling APIs.
- To count the occurrences of each word in the file.
- To store and process words efficiently.
- To display the frequency of each word.

Source Code:

```
import java.io.*;
import java.util.*;

public class Program5 {
    public static void main(String[] args) {
        String fileName = "Sample.txt";

        try {
            File file = new File(fileName);
            Scanner sc = new Scanner(file);

            HashMap<String, Integer> wordCount = new HashMap<>();

            while (sc.hasNext()) {
                String word = sc.next().toLowerCase();
                word = word.replaceAll("[^a-zA-Z0-9]", "");

                if (!word.isEmpty()) {
                    wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
                }
            }
            sc.close();

            System.out.println("=== Word Occurrences in File ===");
            System.out.printf("%-15s %-10s\n", "Word", "Count");
            System.out.println("-----");

            List<String> keys = new ArrayList<>(wordCount.keySet());
            Collections.sort(keys);

            for (String key : keys) {
                System.out.printf("%-15s %-10d\n", key, wordCount.get(key));
            }

        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + fileName);
        }
    }
}
```

Output:

```
=== Word Occurrences in File ===  
Word                Count  
-----  
a                    1  
file                 1  
fun                  1  
hello                2  
is                   2  
java                 2  
sample              1  
this                 1  
world                2
```

➤ Program 6

Problem Statement:

Display all files of a given directory using File class.

Objective:

- To design a Java program to access a directory using the File class.
- To retrieve all files present in the given directory.
- To differentiate between files and subdirectories.
- To display the list of files in the directory.

Source Code:

```
import java.io.File;
import java.text.SimpleDateFormat;
import java.util.Date;

public class Program6 {
    public static void main(String[] args) {
        String dirPath = ".";
        File dir = new File(dirPath);

        if (!dir.exists() || !dir.isDirectory()) {
            System.out.println("Directory not found: " + dirPath);
            return;
        }

        File[] files = dir.listFiles();

        if (files == null || files.length == 0) {
            System.out.println("No files found in directory: " + dirPath);
            return;
        }

        System.out.println("=== Files in Directory: " + dir.getAbsolutePath() + " ===");
        System.out.printf("%-5s %-30s %-10s %-20s\n", "No.", "File Name", "Type", "Last Modified");
        System.out.println("-----");

        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy HH:mm:ss");

        int count = 1;
        for (File f : files) {
            String type = f.isDirectory() ? "Directory" : "File";
            String lastModified = sdf.format(new Date(f.lastModified()));
            System.out.printf("%-5d %-30s %-10s %-20s\n", count++, f.getName(), type, lastModified);
        }
    }
}
```

OutPut:

```
=== Files in Directory: C:\Users\de11\Desktop\Java-Programing-Assignment\Assignment-2\Program-6\. ===
No.  File Name                Type      Last Modified
-----
1    Program6.class            File      26-03-2026 21:02:21
2    Program6.java             File      23-03-2026 06:20:30
```


➤ Program 7

Problem Statement:

Implement TCP Echo Client–Server program

Objective:

- To design a TCP-based client-server program in Java.
- To establish communication between client and server using sockets.
- To send messages from client to server and receive echoed responses.
- To demonstrate data exchange over network using TCP protocol.

Source Code:

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Program7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Select mode: 1 = Server, 2 = Client");
        int mode = sc.nextInt();
        sc.nextLine();
        if (mode == 1) {
            startServer();
        } else if (mode == 2) {
            startClient();
        } else {
            System.out.println("Invalid option.");
        }
    }

    private static void startServer() {
        int port = 5000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("=== TCP Echo Server Started ===");
            System.out.println("Waiting for client on port " + port + "...");

            Socket clientSocket = serverSocket.accept();
            System.out.println("Client connected: " + clientSocket.getInetAddress());

            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

            String message;
            while ((message = in.readLine()) != null) {
                System.out.println("Received from client: " + message);
                out.println("Echo: " + message);
            }
            clientSocket.close();
            System.out.println("Client disconnected.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
private static void startClient() {
    Scanner sc = new Scanner(System.in);
    String serverAddress = "localhost";
    int port = 5000;

    try (Socket socket = new Socket(serverAddress, port)) {
        System.out.println("Connected to server: " + serverAddress);

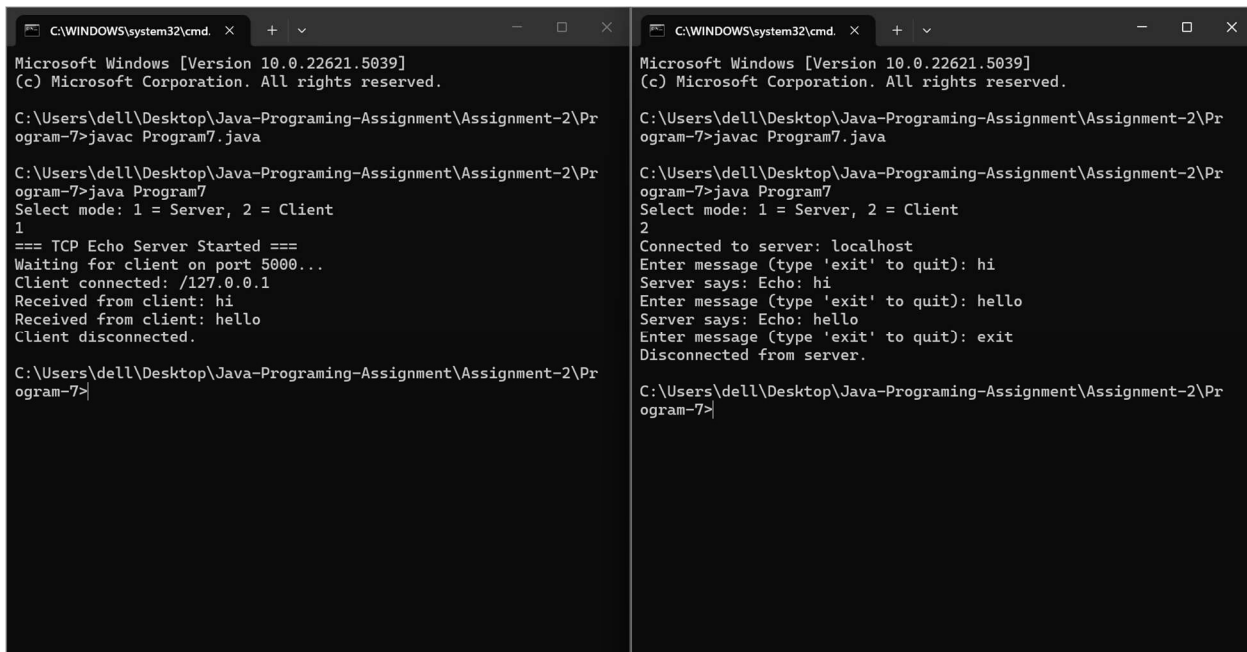
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

        String message;
        while (true) {
            System.out.print("Enter message (type 'exit' to quit): ");
            message = sc.nextLine();
            if (message.equalsIgnoreCase("exit")) break;

            out.println(message);
            String response = in.readLine();
            System.out.println("Server says: " + response);
        }

        System.out.println("Disconnected from server.");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

OutPut:



```
C:\WINDOWS\system32\cmd. X + v - □ X
Microsoft Windows [Version 10.0.22621.5039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-7>javac Program7.java

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-7>java Program7
Select mode: 1 = Server, 2 = Client
1
=== TCP Echo Server Started ===
Waiting for client on port 5000...
Client connected: /127.0.0.1
Received from client: hi
Received from client: hello
Client disconnected.

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-7>
```

```
C:\WINDOWS\system32\cmd. X + v - □ X
Microsoft Windows [Version 10.0.22621.5039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-7>javac Program7.java

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-7>java Program7
Select mode: 1 = Server, 2 = Client
2
Connected to server: localhost
Enter message (type 'exit' to quit): hi
Server says: Echo: hi
Enter message (type 'exit' to quit): hello
Server says: Echo: hello
Enter message (type 'exit' to quit): exit
Disconnected from server.

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-7>
```

➤ Program 8

Problem Statement:

Develop GUI-based Investment Calculator using Swing

Objective:

- To design a GUI-based application using Java Swing.
- To calculate investment returns based on user input.
- To take inputs like principal amount, rate of interest, and time period.
- To display the calculated investment value through an interactive interface.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.text.DecimalFormat;

public class Program8 extends JFrame implements ActionListener {
    private JLabel principalLabel = new JLabel("Principal Amount:");
    private JLabel rateLabel = new JLabel("Annual Interest Rate (%):");
    private JLabel yearsLabel = new JLabel("Number of Years:");
    private JLabel resultLabel = new JLabel("Future Value: ");

    private JTextField principalField = new JTextField();
    private JTextField rateField = new JTextField();
    private JTextField yearsField = new JTextField();

    private JButton calculateButton = new JButton("Calculate");

    public Program8() {
        setTitle("Investment Calculator");
        setSize(450, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10,10,10,10);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        gbc.gridx = 0; gbc.gridy = 0; add(principalLabel, gbc);
        gbc.gridx = 1; gbc.gridy = 0; add(principalField, gbc);

        gbc.gridx = 0; gbc.gridy = 1; add(rateLabel, gbc);
        gbc.gridx = 1; gbc.gridy = 1; add(rateField, gbc);

        gbc.gridx = 0; gbc.gridy = 2; add(yearsLabel, gbc);
        gbc.gridx = 1; gbc.gridy = 2; add(yearsField, gbc);

        gbc.gridx = 0; gbc.gridy = 3; gbc.gridwidth = 2; add(calculateButton, gbc);
```

```
gbc.gridx = 0; gbc.gridy = 4; gbc.gridwidth = 2; add(resultLabel, gbc);

calculateButton.addActionListener(this);
setLocationRelativeTo(null);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    try {
        double principal = Double.parseDouble(principalField.getText());
        double rate = Double.parseDouble(rateField.getText()) / 100;
        int years = Integer.parseInt(yearsField.getText());
        double futureValue = principal * Math.pow(1 + rate, years);
        DecimalFormat df = new DecimalFormat("#.##");
        resultLabel.setText("Future Value: $" + df.format(futureValue));
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Enter valid numbers", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String[] args) {
    new Program8();
}
```

OutPut:

C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-2\Program-8>java Program8

